

Future Enhancement - HRMS Architecture with Pagination, Security & Multi-Language Support

This roadmap ensures:

- ✓ **Scalability** (Microservices, Multi-Database)
 - ✓ **Security** (JWT, SSO, Penetration Testing Compliance)
 - ✓ **Performance & Monitoring** (Logging, Observability)
 - ✓ **Multi-Platform Support** (Web, Mobile, API-First Design)
 - ✓ **Efficient Data Fetching with Pagination** (HRMS.UI + HRMS.API)
 - ✓ **Multi-Language Support** (Localization & Internationalization)
-

1. Pagination Enhancements

1.1 HRMS.UI (Frontend) - Pagination

- HRMS.UI should support frontend pagination.
- Use **React Query** / **DataTables** for efficient data handling.

1.2 HRMS.API (Backend) - Pagination Query Support

- Implement `page` and `pageSize` query parameters.
 - Optimize database queries using **SKIP** and **TAKE**.
-

2. Security Enhancements

2.1 Authentication & Authorization

- **JWT Authentication** for secure API communication.
- **SSO (Single Sign-On)** via **OAuth2.0** or **OpenID Connect**.

2.2 Security Compliance

- **Penetration Test Readiness:** XSS prevention, SQL Injection protection.
 - **Role-Based Access Control (RBAC):** Secure API endpoints.
-

3. API Scalability & Microservices Readiness

- Convert **HRMS.API** into modular microservices:
 - **EmployeeService**
 - **JobService**
 - **NotificationService**
 - **AuthService**

- Implement **Docker + Kubernetes** for containerization.
-

4. Multi-Database Support

- Modify **HRMS.DataAccess** to support:
 - **SQL Server**
 - **PostgreSQL**
 - **MongoDB**
 - Implement **Repository Pattern** for seamless database switching.
-

5. Logging & Monitoring

- Use **Serilog + ELK Stack** for centralized logging.
 - Implement **Prometheus + Grafana** for performance monitoring.
-

6. Cross-Platform Support

- Migrate **HRMS.UI** to **ReactJS** for a modern web experience.
 - Develop a **React Native Mobile App** for multi-device support.
-

7. Multi-Language Support

7.1 UI Localization (HRMS.UI)

- Implement **i18n (Internationalization) Support**.
- Use **React Intl / i18next** for multi-language UI.
- Support language switching dynamically in the application settings.

7.2 API Localization (HRMS.API)

- Implement **Resource Files** (`.resx` for C#) for multi-language support in API responses.
- Allow clients to specify language preferences using **Accept-Language** headers.
- Store translations in **SQL Server / NoSQL (MongoDB) for dynamic updates**.

7.3 Database Localization (HRMS.DataAccess)

- Store multilingual data using **separate translation tables**.
- Implement **Localized Fields in Entities** (e.g., `DepartmentName_EN`, `DepartmentName_FR`).

